第四章　指令系统

※主要内容

(1)指令系统的组成　　(指令的表示)

指令功能，指令格式(组成/信息表示方法)，

操作码编码，指令字组成　　(操作的编码[表示])

(2)操作数(OPD)的存放　　(数据的地址组成)

OPD在REG、MEM、IR中的存放方式、地址表示

(3)寻址方式　　(地址的编码[表示])

指令寻址方式，数据寻址方式，指令格式分析(举例)

(4)指令系统举例 (△)

ISA的数据表示、OPD存放、寻址方式、指令格式、指令功能

(5)指令系统的发展 (△)

CISC，RISC

※总体要求：理解指令所含信息的表示方法，能按给定ISA编程

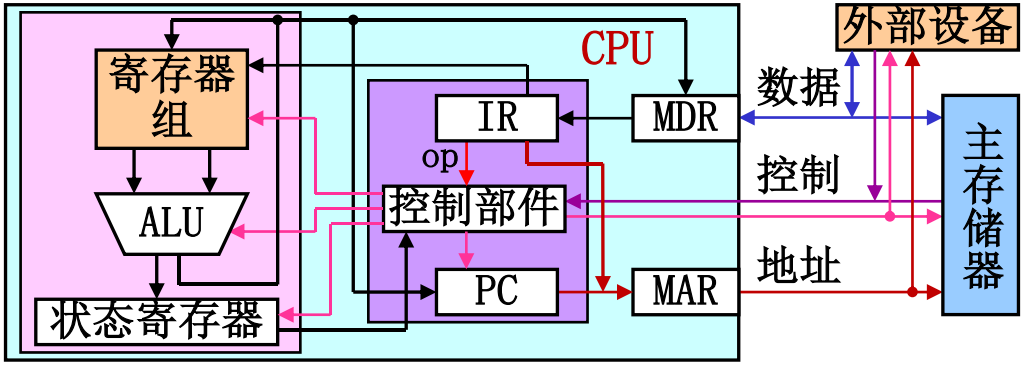
§4.1 指令系统组成

※主要内容：指令功能，指令格式，操作码编码，指令字

*计算机系统组成回顾：

硬件组成—

CPU+主存+外设



软件组成—

程序由指令序列组成；

指令由操作码+地址码组成、用二进制编码表示

200	R[1]←M[204]+2
201	R[1]=0时转203
202	M[2005]←8
203	停机(转800)
204	x
205	flag

工作方式—（软件的执行方式）

循环的指令执行过程；指令执行过程包括：

取指令、分析指令、执行指令、计算指令地址

└←取操作数、操作、写结果

OP	A1	A2

软硬件接口—指令系统，即所有指令的功能-格式的约定

（交界面）

（硬件）（软件）

***指令系统：** —又称指令集(Instruction Set)或ISA

目标—约定所有指令的指令功能及指令格式(表示格式)

内容—包括数据表示、操作数存放方式、寻址方式、指令功能、指令格式等

评价—代码效率(指令字长&指令条数)、执行速度(译码时间&执行时间)

设计—基于评测程序，采用频带分析法，进行统计&选择&设计
(用户提供) (无量化标准) (计算机系统结构的任务)

结果—仅支持高频使用及无法替代的数据类型、操作类型等
└←考虑硬件性能、性/价，其余功能可通过软件转换

风格—有CISC、RISC两种，目前RISC是主流(适于并行处理)
| └←侧重执行速度(功能弱/格式规整)
└←侧重代码效率(功能强/格式复杂)

CISC—Complex Instruction Set Computer
RISC—Reduced Instruction Set Computer

本课程的要求：认知指令系统(本章)、实现指令系统(下章)

一、指令功能

包括数据操作(取操作数+操作+写结果[含标志)、指令地址计算

数据操作例：目的操作数←源操作数1 OP 源操作数2

指令地址计算例：PC←PC+"1" 或 PC←转移目标地址

1、指令的操作数(Operand, OPD)

*硬件支持的数据类型：高频使用的数据类型，有多种长度(存储效率高)

(称为数据表示)

例：RISC-V支持8/16/32位整数，32/64位浮点数，64位SIMD数

思考①：RISC-V中机器字长是？存储器单元的长度是？ 32位，8位

*OPD的数据类型：数据表示的子集(可简化运算部件)

└←指运算类指令的数据

*OPD的存放部件：寄存器/寄存器组、存储器、外设、IR等

具体器件—	寄存器中	存储单元中	地址码中(立即数)
数据地址—	寄存器编号	存储单元地址	无

思考②：CPU中设置寄存器组的好处是？数据寄存器的长度是？

思考③：立即数与常数的区别是？ 指令中是否需编码表示(如x=x+6、i++中的6和1)

思考④：指令的存放部件是？ 存储器(存储程序工作方式所规定)

2、指令的操作(Operation, OP)

***指令功能的描述：** (有多种方法)

数据操作— ①功能为"目的OPD←源OPD1 OP 源OPD2"， ←直观/不规范

源OPD为部件中内容，目的OPD为部件地址，OP为C语言运算符

②汇编语言指令格式 ←规范/不直观

OPD—

	a#寄存器	b#存储器单元	立即数	外设端口
地址描述	Ra 或R[a]	M[b]	/	以后
内容描述	(Ra)或R[a]	M[b]	真值	再说

示例—

R0←(R1) 或R[0]←R[1] 源OPD在1#REG中，目的OPD在0#REG中

R0←M[0100H] 或R[0]←M[0100H] 源OPD在0100H#存储单元中，目的OPD在0#REG中

M[0100H]←(R2) 或M[0100H]←R[2] 源OPD在2#REG中，目的OPD在0100H#存储单元中

R0←(R2)+6 或R[0]←R[2]+6 源OPD在2#REG及IR中，目的OPD在0#REG中

指令地址计算— 顺序型指令隐含为PC←(PC)+"1"， ←"1"指所占存储单元个数

转移型指令描述为PC←表达式 ←与数据操作合并

如：(R0)=0时PC←PC+40，否则PC←PC+"1"

***常见指令类型：** 数据传送、算逻运算、转移控制等 ←均高频使用

PPT约定：寄存器记为REG(Register)，存储器记为MEM(Memory)

*数据传送指令：

PPT约定：寄存器记为REG(Register)，存储器记为MEM(Memory)

数据操作—REG、MEM、外设间的数据传送(源OPD、目的OPD各1个)

如：R0←(R1)　　←1#REG的内容传送到0# REG中

R0←M[0100H]　　←0100H#存储单元的内容传送到0#REG中

M[0100H]←(R2)　　←2#REG的内容传送到0100H#存储单元中

	寄存器传送 (REG←REG)	取数 (REG←MEM)	存数 (MEM←REG)	输入 (REG←IO)	输出 (IO←REG)
源OPD	n位	m位①	n位③或m位	m位	n位或m位
目的OPD	n位	n位②或m位	m位	n位或m位	m位
结果标志	/	/④	/	/	/

注：①MEM中数据长度m有多种(可提高存储效率)

②REG长度＝机器字长n，RISC为n(简化硬件)、m＜n时先进行位扩展

③RISC为n、m＜n时先进行位截断　　④无运算所致

思考：若REG长度为16位、存储单元长度为8位，R0←M[0100H]传送的单元是？

M[0100H]和M[0101H](长数据连续存放)，仅传送1个单元时为R0←ZExt(M[0100H])

指令地址计算— PC←(PC)＋"1"

←顺序型指令均隐含表示

*算术逻辑运算指令：

数据操作—OPD为整数(有/无符号、位串)， 需保存结果标志

如：R0←(R1)＋M[0100H]、R2←(R2)＋1；R0←(R0)&(R1)、R0←~(R0)

	＋/－	×/÷	++/--/求补	按位的&/|/~
源OPD	2个，n位①	n位	1个，n位	2个/1个，n位
目的OPD	n位	n位或2n位	n位	n位
结果标志	ZF/CF/OF/SF	CF/OF	ZF/CF/OF/SF	ZF
注：①RISC为机器字长，CISC有多种长度				

思考①：n位ALU如何支持m位运算？ 结果在低位，标志按m位产生(有多种控制信号)

指令地址计算—PC←(PC)＋"1"

思考②：标志的存放部件是？

*移位运算指令：

无算术左移指令！

数据操作—逻辑左移、逻辑右移、算术右移等，

如：R0←(R0)<<2、R1←(R1)>>$_L$3、R1←(R1)>>$_A$(R2)

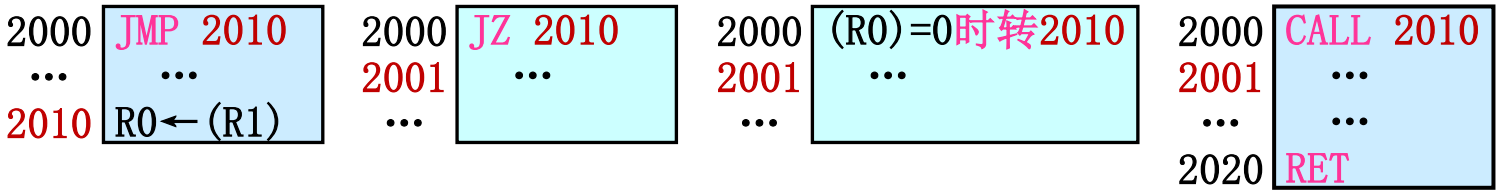
源OPD为n位(数据)、$\log_2 n$位(移动位数)，

结果标志为CF/OF

指令地址计算—PC←(PC)＋"1"

*转移控制指令：

转移类型— 跳转(无条件转移)、分支(条件转移)、调用、返回



数据操作— 产生数据状态、判断是否转移，仅分支需要

- 产生数据状态：源OPD为2个数据或标志，操作为减法或无（RISC采用）
- 判断是否转移：基于转移条件(如>/≥等)，方法有多种（CISC采用）

指令地址计算—

下条指令地址： PC＝计算结果 或 PC＝(PC)＋"1"

- PC＝计算结果：指令寻址方式中讨论
- PC＝(PC)＋"1"：仅分支有

地址源OPD： 转移目标地址，或相对偏移量(如PC＝(PC)+disp)

思考①：CALL、RET指令的返回地址如何表示？如何支持嵌套调用？

隐含表示，保存在栈中

*其他指令：

浮点运算—浮点算术运算指令等

十进制运算—加法校正、减法校正等　　←硬件为二进制运算

CPU控制—标志(如CF/ZF)置位、标志复位、程序结束等

※指令功能小结：

功能组成— 数据操作(取OPD+操作+写结果[含标志])、指令地址计算

数据操作— OPD存放部件有多种(REG/MEM/IR等)、常按地址访问，

操作类型有多种(区分OPD类型)、源OPD个数不同，

所存目的OPD相关于应用、所存标志取决于操作类型

(如n位乘积)　　(如传送不产生标志)

指令地址计算— 顺序型指令：NextPC＝(PC)＋"1"

转移型指令：NextPC＝运算结果

二、指令格式 --指令功能的表示

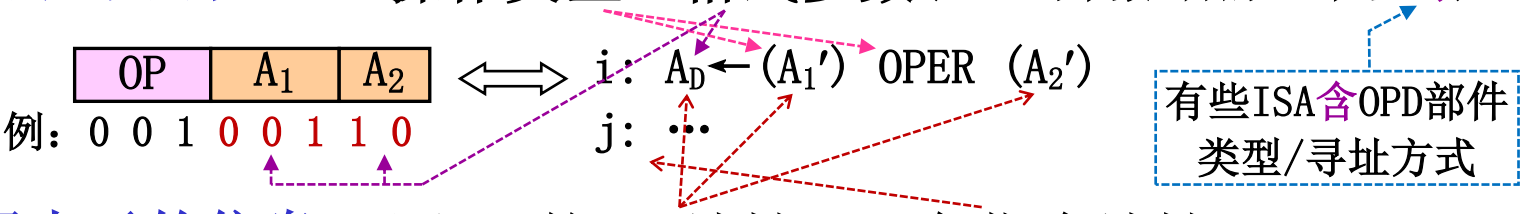
用编码表示指令功能所含信息(操作类型、源OPD/目的OPD、下条指令地址)

*指令格式的组成：



不同指令的标识—用操作码实现(要求编码无二义性)

操作码表示的信息—操作类型、格式参数(地址码个数+目的OPD位置等)



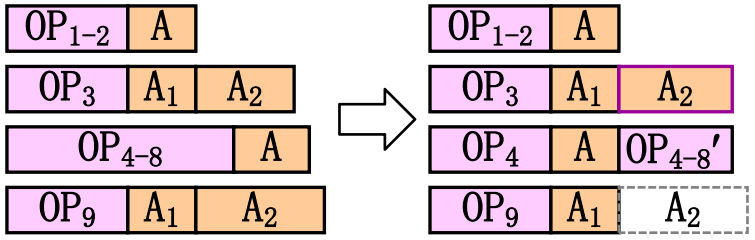
地址码表示的信息—源/目的OPD地址、下条指令地址

*指令中信息的表示：

性能指标—平均码长、规整性 ←影响存储空间、译码时间

表示方法—显式(编码)、隐式(约定)

操作码可指明



1、操作码

需表示的信息—操作类型(功能+OPD类型)、格式参数(个数+位置等)

*需显式表示的信息：操作功能须显式表示，其余可隐式表示

隐式表示的条件—同一指令中某信息仅1种取值时

如：指令的OPD类型仅为32位无符号整数时，
指令的地址码个数仅1个时

*操作码的编码方法：（面向所有指令，尽量规整[相同属性约定相同]）

定长编码—按序编码，长度=$\lceil \log_2(\text{显式表示信息的种类数}) \rceil$

变长编码—带权编码，如扩展编码(基于哈夫曼编码)

k叉哈夫曼树→┴←提高规整性←┘ └←2叉哈夫曼树

例1：定长编码、哈夫曼编码、扩展编码示例 （下页）

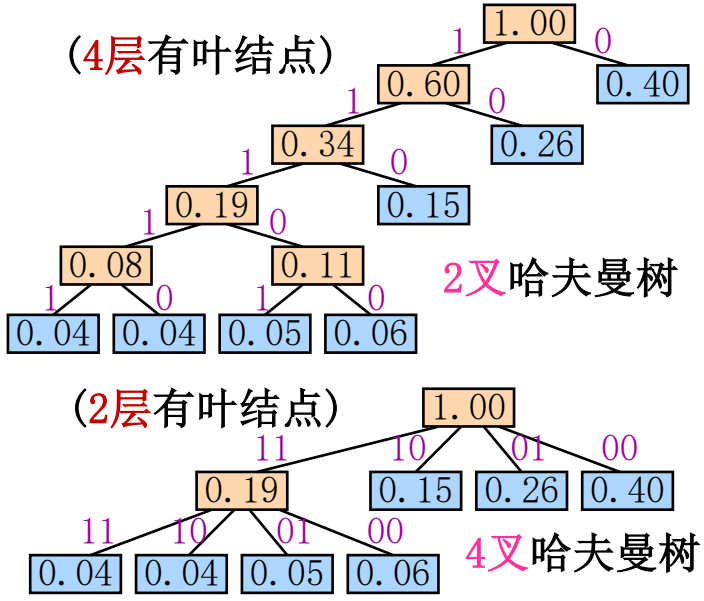
*操作码的特性：不同指令的编码无二义性



└←区分不同指令的操作、隐含表示的信息
(操作类型、源/目的OPD、指令地址计算方法)

例1：7种操作的使用频率分别为0.4、0.26、0.15、0.06、0.05、0.04、0.04，分别按定长、哈夫曼、扩展(2种长度)格式进行编码

操作I_i	频率P_i	定长码	哈夫曼码	扩展码
I1	0.40	000	0	00
I2	0.26	001	10	01
I3	0.15	010	110	10
I4	0.06	011	11100	11 00
I5	0.05	100	11101	11 01
I6	0.04	101	11110	11 10
I7	0.04	110	11111	11 11
平均码长($\sum P_i I_i$)		3.00	2.32	2.38
规整性(码长种类)		1种	4种	2种



扩展编码的要求—扩展标志与短编码间无二义性

反例：有2个人(王二、王二宁)，“王二宁静一会儿”的目标是谁？

思考：写出23种操作(7个频率较高、16个频率较低)的扩展编码(2种长度)

高频操作：000~110

低频操作：111 | 0000~1111

2、地址码

需表示的信息—源OPD地址、目的OPD地址，下条指令地址

*需显式表示的信息：均可隐式表示(操作码可指明)

如：加1指令的1隐含表示，
RET指令的转移目标地址隐含表示

*地址码的编码方法：（面向一条指令，尽量有共性[相同属性约定相同]）

每个地址用形成参数(一个/多个)表示 ←数据寻址方式中讨论

例2：假设CPU的寄存器组有8个寄存器(长度为16位)，存储器有256个存储单元(按字节编址)，分析下列指令的地址码、下条指令地址

	主存	指令功能
20	00100010	取8位数据，
21	00101000	R2←ZExt(M[28H])
22	00111010	取16位数据，
23	00100110	R2←M[(R1)+6]
24	01000010	R2←(R2)+1

地址长度：REG地址为3位，MEM地址为8位

地址码：REG为直接表示，常数为隐含表示，
MEM为直接或基址表示

下条指令地址：(PC)+2、(PC)+2、(PC)+1

操作码：4位或5位，标识不同指令

*地址码的特性：仅表示当前指令的OPD地址/OPD

└←不同指令可能不同(如存数的MEM地址码可能就1种)

3、指令字

即指令格式的实例(编码固定) ←仅包含显式表示信息的编码

*指令字长：指令对应指令格式的位数

要求—指令字长＝n×主存单元长度 ←机器字长＝m×主存单元长度，m为常数

*术语：单地址指令、双地址指令、零地址指令 ←指OPD地址个数

单字长指令、双字长指令、半字长指令 ←字长指机器字长

*指令集结构：

定长指令字结构—所有指令的指令字长完全相同(n为常数)

变长指令字结构—所有指令的指令字长不尽相同(n可变)

例2：设指令长度为16位，有零地址指令、单地址指令、双地址指令3种指令格式，每个地址码均为6位。

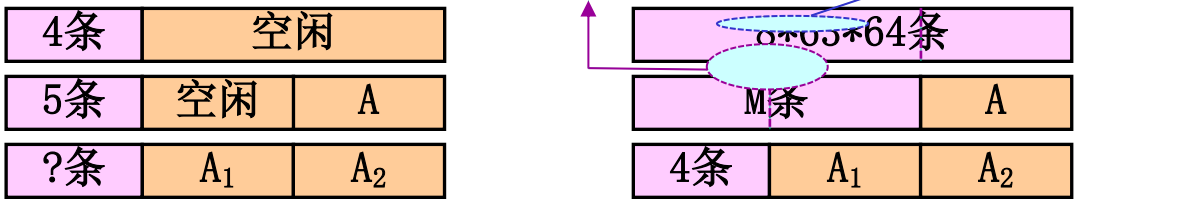
⑴操作码采用定长编码方式时，若零地址指令、单地址指令分别为4条和5条，则双地址指令最多可定义多少条(N)？

⑵操作码采用扩展编码方式时，若零地址指令、双地址指令分别为8*63*64条和4条，单地址指令最多可定义多少条(M)？

每条指令占1个操作码　　操作码占满空闲位时

解：⑴ 操作码长度＝16-6-6＝4，N≤2^4-(4+5)

≤$(2^4-4)\times 2^6$条　　=⌈8*63*64/2^6⌉条

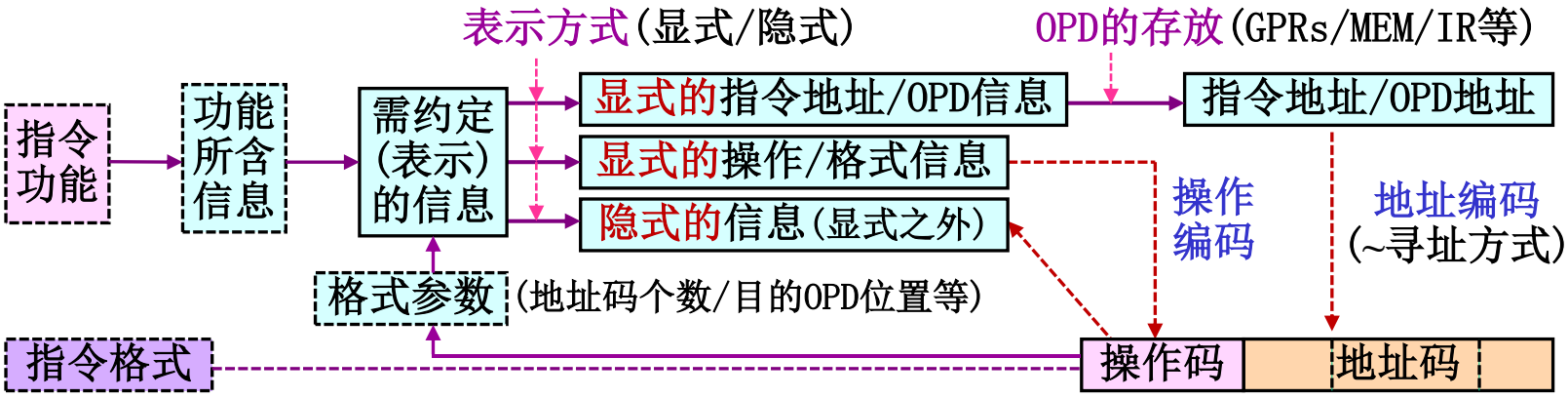


⑵双、单、零地址指令操作码长度分别≤4、10、16，

有M+⌈8*63*64/2^6⌉≤$(2^4-4)\times 2^6$，即M≤12×2^6-8*63

※指令格式小结：

指令格式-指令功能的关系— （指令所含信息的表示方法）



注：指令所含信息—操作类型(功能+OPD类型)、源/目的OPD、下条指令地址

操作码的编码—无二义性(标识不同指令)，可指明(当前指令中的)隐式信息

思考—信息隐式表示的条件？必须显式表示的信息？隐式信息如何识别？

指令格式与指令的关系—

一条指令：操作码相同的指令，多条指令可共用一种格式

└←理论上为操作类型，部分ISA包含寻址方式

指令字长：＝对应指令格式长度＝n×主存单元长度

指令集结构：所有指令字长的类型(定长/变长)

§4.2 操作数的存放

※主要内容：数据在REG、MEM、IR中的存放方式、长度及地址表示

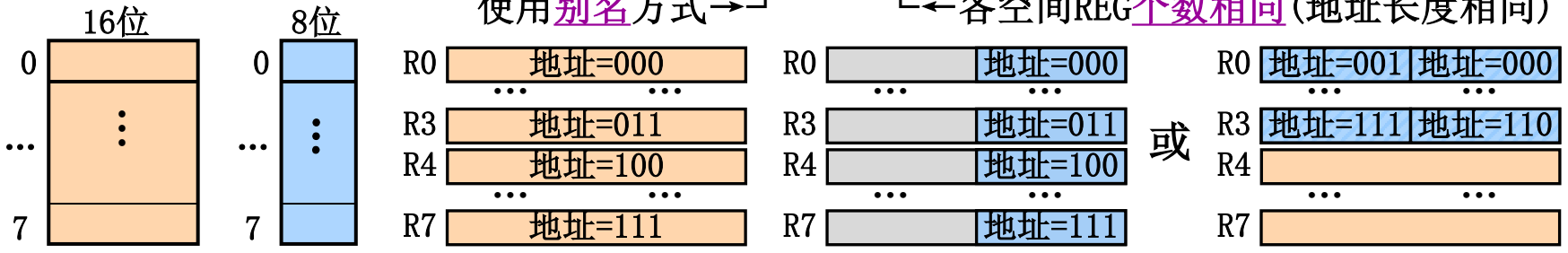
1、数据在REG中的存放 （以定点数为例）

背景—数据有多种类型&长度(如8/16/32位)， ALU支持少量长度(如16/32位)

需求—REG需存放与OPD等长的数据(如16/32位) ←便于直接读/写

*REG的组织：每类有1个REG组，映射为多个REG组空间 ←多个REG组成本高

使用别名方式→ ←各空间REG个数相同(地址长度相同)

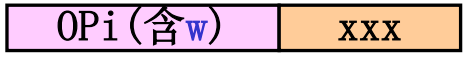


*存放方式：占用某个空间的1个REG(物理REG的全部/部分) ←要求REG可分段读写

如：指令i: 0010(32b加) | 000 ¦ 001 使用R0、R1

指令j: 0111(16b减) | 110 ¦ 111 使用R3的低端和高端

*长度及地址的表示：数据长度—由操作码指明 ←长度∈数据表示

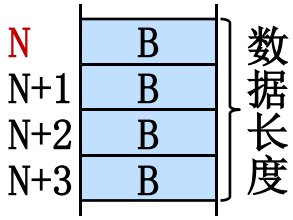


w表示OPD长度，可隐含

数据地址—相应REG组空间的地址

2、数据在MEM中的存放

背景—数据有多种长度，MEM单元长度＝最短数据长度



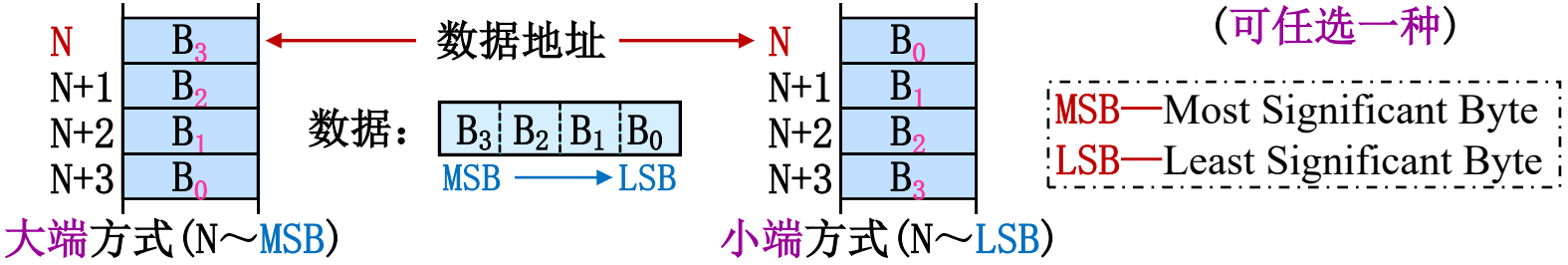
*存放方式： 存放在连续的存储单元中

*长度及地址的表示： 数据长度— 由操作码指明(同REG数据长度)

数据地址—用最小单元地址(N)表示 ←加易实现

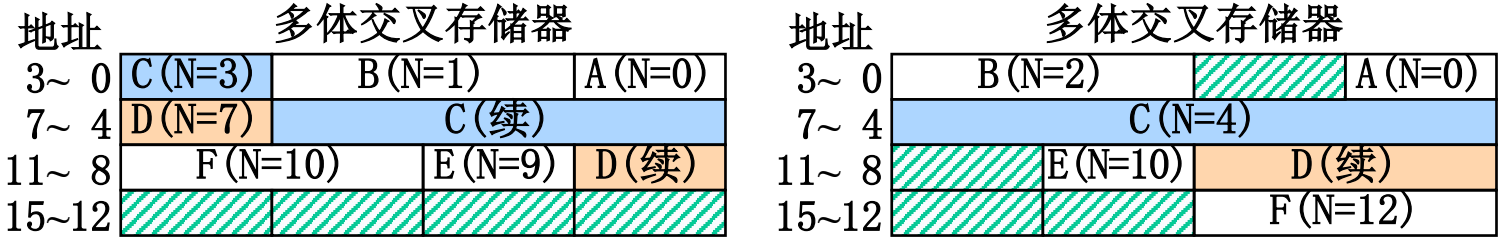
*存放方式的属性： 包括端序(Endian)、对齐(Alignment)

端序—存放时的字节顺序(即N中内容)，有大端、小端2种



对齐—存放时的位置限制(即N的取值)，有不对齐、对齐2种

类型有边界对齐、4字节对齐等←┘ (目前均为对齐)



边界对齐方式： N＝k×数据长度(所占单元数S)　←即N的低$\log_2 S$位=0

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
单元		A	B_0	B_1	C_0	C_1	C_0	C_1	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7			E_0	E_1

4字节对齐方式： ≤4字节的边界对齐方式

例1：若MEM按字节编址，数据在MEM中采用小端、边界对齐方式存放，则2005H#单元可存放机器数12345678H中哪个字节？

解：地址N＝⌊2005H/4⌋*4＝2004H，2005H#单元存放的是 56H

例2：32位计算机的MEM按字节编址、数据采用大端、边界对齐方式存放。对C语言结构数组：struct ST{ char a;int b;short c;} ss[20];
若ss在MEM中的起始地址为200H，则ss[1].b的地址是多少？

解：ss存放效果如右图，则p＝204H，q＝208H；
为使ss[i]的各元素都能边界对齐，
则sizeof(ST)＝常数＝m×sizeof(int)，
故k＝208H＋⌈2/4⌉×4＝20CH，x＝210H

200H	ss[0].a	char
p	ss[0].b	int
q	ss[0].c	short
k	ss[1].a	char
x	ss[1].b	int
y	ss[1].c	short

思考：为减小所占存储空间，编译程序应如何优化？

3、数据在其他部件中的存放

***数据在IR中的存放：**



背景—数据有多种长度，常≤OPD长度　　←便于缩短指令字长

存放方式—连续存放(端序同MEM、不对齐)，或按约定格式存放

无地址　　←指令格式已约定位置

***数据在堆栈中的存放：**

背景—数据仅一种长度，常＝机器字长　　←应用需求如此，可简化实现

存放方式—连续存放(端序同MEM、对齐)，无地址　　←便于统一处理

└←操作码控制栈顶指针变化

※数据存放方式小结：　(决定了数据的地址组成、存取组织)

存放部件	存放方式		数据地址	长度表示
REG	占用全部/部分空间		REG编号	操作码指明
MEM	端序(大/小)	对齐(目前)	最小单元地址	
IR	端序(同MEM)	不对齐		
堆栈	端序(同MEM)	对齐		机器字长

§4.3 寻址方式

※主要内容：指令寻址方式，数据寻址方式，指令格式分析

*寻址方式：根据地址码形成OPD/指令地址的方法，即地址＝f(地址码)

目标—编码时缩短地址码长度，执行时方便地址形成 ←函数f简单

*OPD/指令地址的类型回顾：

思考①：REG地址位数会与MEM地址相同吗？

OPD地址—REG地址、MEM地址、IO端口地址 ←立即数、栈中数据无地址

指令地址—MEM地址 ←存储程序工作方式要求

*软/硬件中的OPD/指令地址：（IO端口地址以后再讨论）

REG地址—位数相同(与软件需求无关/较小)，硬件直接使用(实现简单)

MEM地址—位数不同(软件要求很大、硬件考虑成本)，硬件变换后使用

ISA约定→ ←设计者约定 ←基于装入位置

思考②：软硬件大小不一致，程序如何执行？

每次装入部分程序(局部性)，
访问时先进行地址变换(没办法)

有效地址(EA)：软件中的MEM地址(逻辑地址/虚拟地址)，根据地址码形成

物理地址(PA)：硬件中的MEM地址

1、指令寻址方式

***指令类型与下条指令地址：**

顺序型指令—EA＝(PC)＋"1"，均隐式表示(EA与指令内容无关)

转移型指令—EA＝f(地址码)，常显式表示(EA与指令内容有关)

***指令的寻址方式：** 有隐含、直接、相对等 ←数据寻址中一并讨论

***下条指令地址的形成：**

寻址方式的识别—



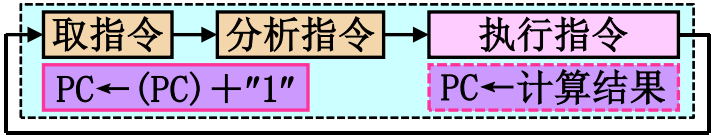
指令执行过程的理论方案

分析指令时，根据操作码判断(CISC常需地址码参与)

下条指令地址的形成—

取指令时，可实现顺序寻址(EA＝(PC)＋"1") ←可用计数器实现

执行指令时，实现跳跃寻址(EA＝f(地址码)) ←需等待转移判断



指令执行过程的实际方案

思考：什么指令字结构，最利于取指令时实现顺序寻址？

2、数据寻址方式

*寻址方式的识别：指令译码时，通过操作码/地址码区分(CISC为后者)

*地址码的组成：

地址码：寻址方式位F | 地址参数A

OPD或EA＝f(F, 地址参数)

例：RD←(RD)＋(RS)

0 1 0	RD	0	RS

RD←(RD)＋M[(RS)]

0 1 0	RD	1	RS

同一指令的同一OPD可以有多种寻址方式(仅1种时F可缺省)

*立即寻址(Immediate Addressing)方式：

OPD放在IR中，地址参数为OPD本身(不是OPD地址)



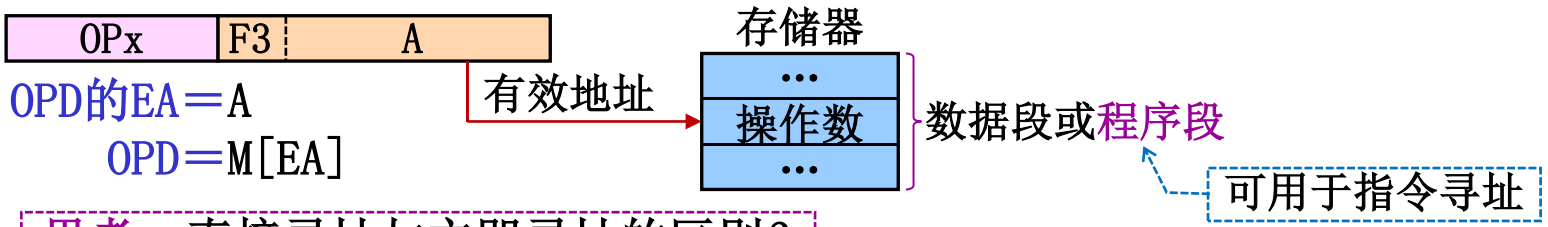
*寄存器寻址(Register Addressing)方式：

OPD放在数据REG中，地址参数为数据REG号



*直接寻址(Direct Addressing)方式：

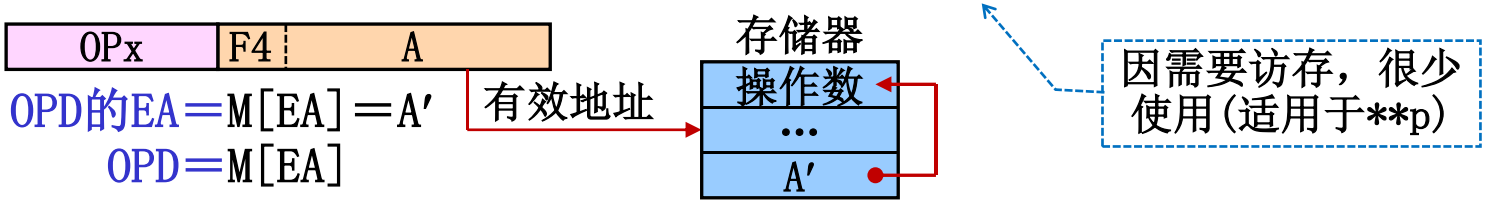
OPD放在MEM中，地址参数为OPD的EA



思考：直接寻址与立即寻址的区别？

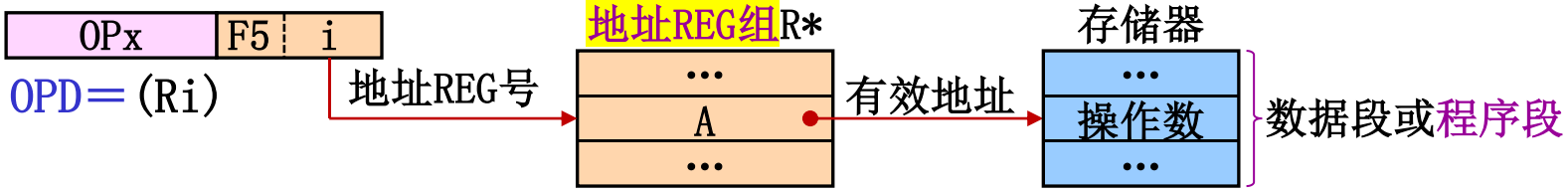
*间接寻址(Indirect Addressing)方式：

OPD放在MEM中，地址参数为存放OPD的EA的EA



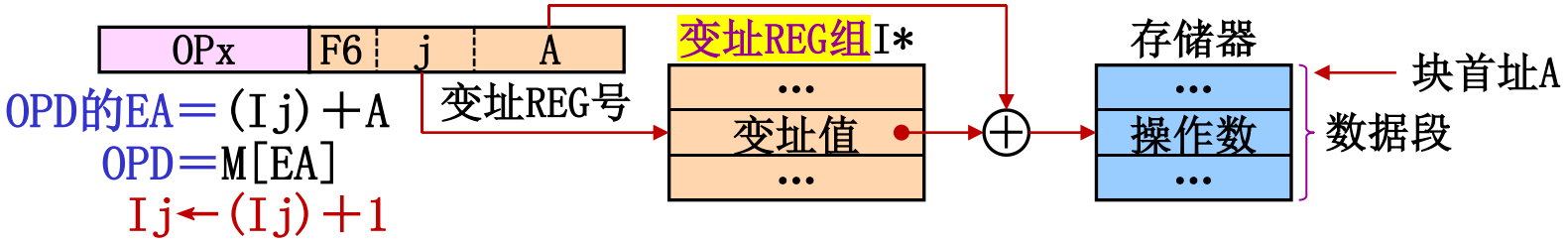
*寄存器间接(Register Indirect Addressing)寻址方式：

OPD放在MEM中，地址参数为存放OPD的EA的地址REG号



*变址寻址(Indexing Addressing)方式：

OPD放在MEM中，地址参数为变址REG号及形式地址(基准地址)

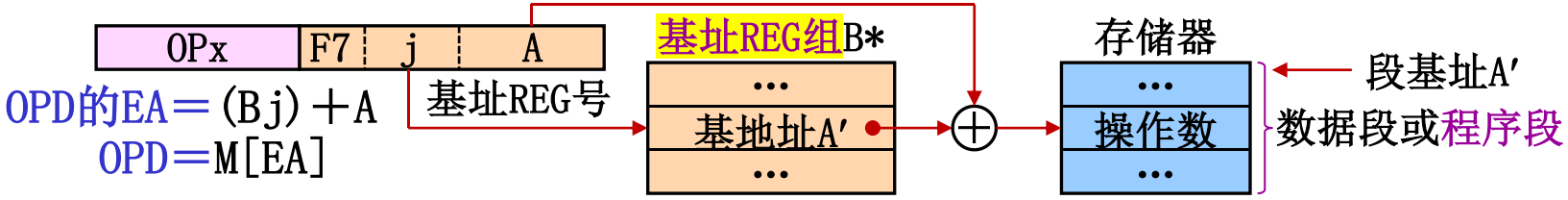


特点—常用于数据块访问，变址REG的值自动改变(每次使用时)

*基址寻址(Base Addressing)方式：

(Bj)为基准地址

OPD放在MEM中，地址参数为基址REG号及形式地址(偏移地址)



特点—常用于段式存储管理

说明—基址REG组/变址REG组仅含1个REG时，REG号j需隐式表示

编码恒为0

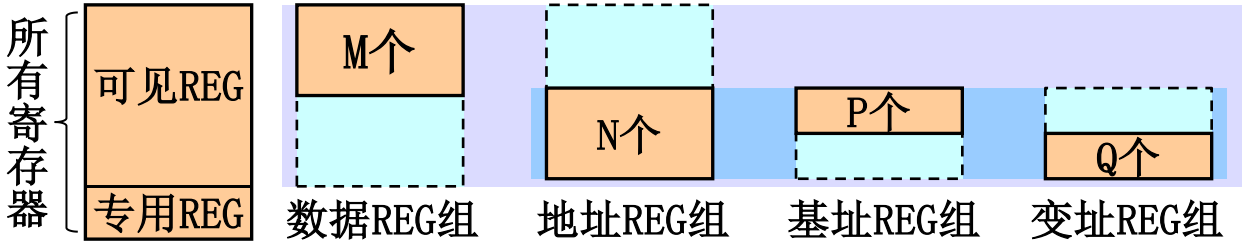
思考：变址寻址与基址寻址的根本区别？

寄存器的常见组织— (理论上数据REG/地址REG/基址REG/变址REG间无关联)

①基址REG、变址REG常为地址REG的子集 ←降低成本

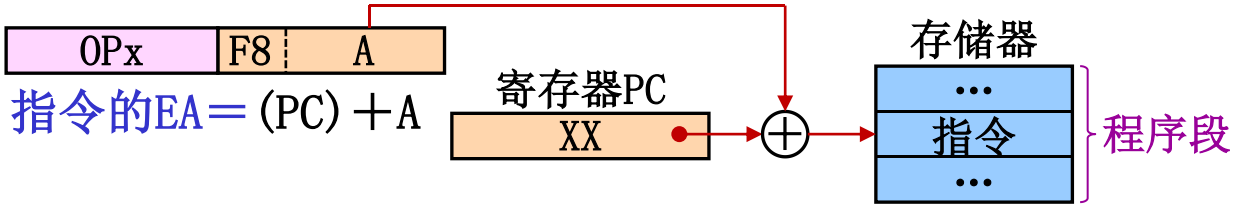
└←具有变址功能，指令指明是否变址

②数据REG与地址REG常混用(称为通用寄存器GPR) ←降低成本



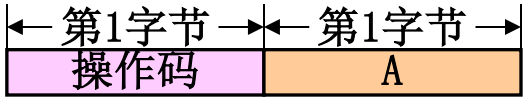
*相对寻址(Relative Addressing)方式：

指令放在MEM中，地址参数为形式地址(偏移量)



说明—相对寻址的A为有符号数！变址/基址寻址的A常为无符号数

例1：某计算机的MEM按字节编址，相对寻址跳转指令格式如下图所示，偏移量A用补码表示。CPU取指令时自动完成PC←(PC)＋"1"。某相对寻址的跳转指令存放在0100H起的存储单元中。



⑴若该指令的转移目标地址为0120H，则M[0101H]＝？

⑵若M[0101H]＝EEH，则该指令的转移目标地址为多少？

解：⑴取指令时，(PC)＝0100H＋2＝0102H；　←指令执行过程实际方案

指令执行时，转移目标地址＝(PC)＋disp，

$[disp]_{补}$＝0120H－0102H＝001EH，

故M[0101H]＝Cut($[disp]_{补}$)＝1EH

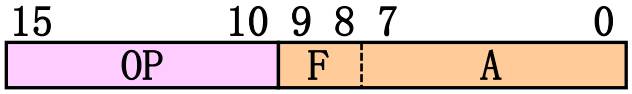
⑵取指令时，(PC)＝0102H；

执行指令时，$[disp]_{补}$＝SExt(EEH)＝FFEEH，

转移目标地址＝(PC)＋disp

＝0102H＋FFEEH＝00F0H

例2：某计算机主存按字节编址，有符号整数用补码表示，单地址指令格式如下，基址REG和变址REG各有1个(记为RB和RI)。设(RB)=8000H、(RI)=0007H、(PC)=1234H，CPU取指令时自动完成PC←(PC)＋"1"，请计算下列指令字的OPD或OPD/指令的EA。



F＝00为立即寻址，01为相对寻址
10为基址寻址，11为变址寻址

(1)4428H　(2)2244H

(3)1330H　(4)3592H

解：(1)4428H＝0100 0100 0010 1000B，$[OPD]_{补}$＝28H

(2)2244H＝0010 0010 0100 0100B，基址寻址(A=44H)，

OPD的EA＝(RB)＋A＝8000H＋0044H＝8044H

(3)1330H＝0001 0011 0011 0000B，变址寻址(A=30H)，

OPD的EA＝(RI)＋A＝0007H＋0030H＝0037H

(4)3592H＝0011 0101 1001 0010B，相对寻址(A=92H)，

$[disp]_{补}$＝SExt(92H)＝FF92H，

转移目标地址＝(PC)＋disp＝(1234H＋2)＋FF92H＝11C8H

*隐含寻址(Implied Addressing)方式：

无地址码，地址形成方法由操作码隐含指明

└←地址参数固定(仅1种取值)

示例：(a)OPD为恒定值，如Ra←(Ra)＋1中的1；

└←立即数不能隐含(采用立即寻址方式)

(b)OPD在栈顶或固定的REG中，如返回指令、顺序寻址

※寻址方式小结：

①寻址方式与编码方式互逆，EA＝*f*(方式位,地址参数)

②数据寻址中的部分MEM寻址方式，可用于指令寻址

	常用寻址方式
数据寻址	除相对寻址外，其余均可使用
指令寻址	常使用直接、相对、寄存器间接、隐含方式

注意—不同ISA的寻址方式命名可能不同

如变址寻址、基址寻址、相对寻址可统称为偏移寻址

3、指令格式分析及其应用

***指令系统分析：** 数据表示仅1种(定点整数)， 寄存器为GPR(通用寄存器)

指令功能		指令字格式	指令寻址方式
赋值(MOV)：	RD←Imm	0000 RD 00 Imm	隐含EA=(PC)＋2
取数(LD) ：	RD←M[(RS)]	0010 RD RS	隐含EA=(PC)＋1
存数(ST) ：	M[(RS)]←(RD)	0011 RD RS	隐含EA=(PC)＋1
加法(ADD)：	RD←(RD)＋(RS)或	010 0 RD RS	隐含EA=(PC)＋1
	RD←(RD)＋M[(RS)]	010 1 RD RS	隐含EA=(PC)＋1
减法(SUB)：	RD←(RD)－(RS)	0110 RD RS	隐含EA=(PC)＋1
自增(INC)：	RD←(RD)＋1	1000 RD 00	隐含EA=(PC)＋1
自减(DEC)：	RD←(RD)－1	1001 RD 00	隐含EA=(PC)＋1
分支(JNZ)：	ZF＝0时PC←Addr	110 0 0000 Addr	直接EA=Addr或隐含EA=(PC)＋2
	或PC←(PC)＋Disp	110 1 Disp	相对EA=(PC)＋Disp
		4位 4位 ←8位→	或隐含EA=(PC)＋1

数据的表示—OPD长度为8位，机器字长为8位

数据的存放—REG有4个、长度为8位，MEM编址单位为8位、地址为8位

操作码编码—采用扩展编码方式，格式信息用隐式方式表示

地址码编码—数据寻址有立即/REG/REG间接/隐含方式，ADD/JNZ指令方式位显式

指令字特征—包含单/双地址指令、单/双字长指令，采用变长指令字结构

*指令系统应用：

例3：若上页指令系统中，有符号整数用补码表示，CPU取指令时自动完成PC←(PC)＋"1"。

⑴说明指令字84H的功能。

⑵写出x=x+3的指令序列，x的地址放在寄存器R0中。

⑶若操作码只可放在高4位，最多还可定义多少条指令？

解：⑴84H＝10000100B，指令功能为R1←(R1)＋1

⑵指令序列为：00000100 00000011 // R1←03H

01010100 // R1←(R1)＋M[(R0)]

00110100 // M[(R0)]←(R1)

⑶高4位已使用10个编码，最多还可定义$2^4-10=6$条指令

§4.4 指令系统举例

※主要内容：MIPS32、IA32

思考：指令系统主要包含的内容？ 数据表示、OPD存放、寻址方式、指令功能、指令格式

1、MIPS32指令系统

*数据表示：

整数—二进制、定点格式、无符号编码&补码，8/16/32位

浮点数—IEEE 754格式，32/64位(单精度/双精度)

*OPD存放：

存储器—按字节编址、32位地址，大端、对齐方式

IR—只存放16位立即数，按约定格式

ALU仅32位运算

寄存器—GPRs：32个×32位，只存放32位OPD(存取时先转换类型)

约定：R0≡0，R31过程调用返回地址，R29栈指针，…

FPRs：32个×32位，双精度OPD占用相邻2个FPR

状态REG：不保存结果标志

*寻址方式：

寻址方式	地址形成方法	注释
立即寻址	OPD＝imm	imm为16位立即数(有符号)
寄存器寻址	OPD＝(rx)	rx为GPRs编号
基址寻址	EA＝(rx)＋imm	相加时先位扩展
PC相对寻址	EA＝(PC)＋imm<<2	
伪直接寻址	EA＝$(PC)_{高4位}$ ‖ addr<<2	Addr为26位地址，‖表示内容拼接
说明：GPRs中，R0≡0，R29栈指针，R30帧指针，R31过程调用返回地址		

指令字长为32位，地址码仅26位

数据寻址方式— 立即、寄存器、基址

思考①：数据的直接寻址、REG间接寻址如何实现？ $0+disp、rx+0

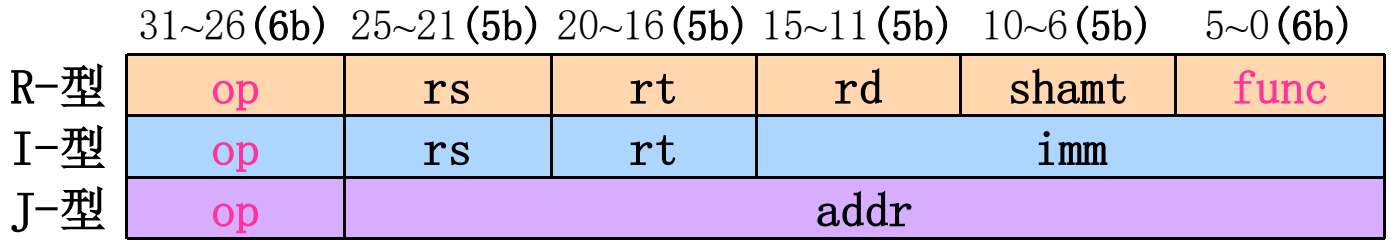
指令寻址方式— PC相对、伪直接，隐含(EA＝(PC)＋4等)

思考②：为何相对寻址中disp要<<2？
为何基址寻址中disp不<<2？

指令字4B对齐,寻址范围可扩大4倍
短数据为8位，按1B边界对齐存放

编译时丢弃低2位，执行时补上低2位

*指令格式：有3种，采用定长指令字结构(32位)



注：op及func—操作码，rs、rt及rd—寄存器号(源及目的OPD)，
shamt—移位位数，imm—立即数或偏移量，addr—形式地址

*指令功能：

常用指令有31条，包括数据传送、ALU运算、转移控制

R-型指令—算术、逻辑、移位，小于置位、过程返回

└←不保存结果标志　　←┴←比较结果(0/1)保存到GPR

I-型指令—加法、逻辑、赋值，存取，小于置位、分支

判段GPR值是否为0→┴→支持多种条件

J-型指令—跳转、过程调用

名称	功能	op	rs	rt	rd	shamt	func
有符号加add	rd←(rs)＋(rt)	000000	rs	rt	rd	00000	100000
无符号加addu	rd←(rs)＋(rt)	000000	rs	rt	rd	00000	100001
有符号减sub	rd←(rs)－(rt)	000000	rs	rt	rd	00000	100010
无符号减subu	rd←(rs)－(rt)	000000	rs	rt	rd	00000	100011
与and	rd←(rs)& (rt)	000000	rs	rt	rd	00000	100100
或or	rd←(rs)| (rt)	000000	rs	rt	rd	00000	100101
异或xor	rd←(rs)^ (rt)	000000	rs	rt	rd	00000	100110
或非nor	rd←~((rs)|(rt))	000000	rs	rt	rd	00000	100111
小于置位slt	rd＝(rs)$<_{有}$(rt)? 1:0	000000	rs	rt	rd	00000	101010
无符号小于置位sltu	rd＝(rs)$<_{无}$(rt)? 1:0	000000	rs	rt	rd	00000	101011
逻辑左移sll	rd←(rt)<<shamt	000000	00000	rt	rd	shamt	000000
逻辑右移srl	rd←(rt)$>>_L$ shamt	000000	00000	rt	rd	shamt	000010
算术右移sra	rd←(rt)$>>_A$ shamt	000000	00000	rt	rd	shamt	000011
逻辑左移sllv	rd←(rt)<<(rs)	000000	rs	rt	rd	00000	000100
逻辑右移srlv	rd←(rt)$>>_L$(rs)	000000	rs	rt	rd	00000	000110
算术右移srav	rd←(rt)$>>_A$(rs)	000000	rs	rt	rd	00000	000111
过程返回jr	pc←(rs)	000000	rs	00000	00000	00000	001000

思考①：为何要区分有/无符号运算？ 无结果标志，便于判断溢出(有符号运算)

思考②：为何没有$<<_A$指令？逻辑非功能如何实现？ 同$<<_L$指令，使用~(rt|$0)

名称	功能	op	rs	rt	Imme
有符号加addi	rt←(rs)＋SExt(imm)	001000	rs	rt	imm
无符号加addiu	rt←(rs)＋SExt(imm)	001001	rs	rt	imm
与andi	rt←(rs)& ZExt(imm)	001100	rs	rt	imm
或ori	rt←(rs)| ZExt(imm)	001101	rs	rt	imm
异或xori	rt←(rs)^ ZExt(imm)	001110	rs	rt	imm
高位取立即数lui	rt←imm<<16	001111	00000	rt	imm
取字lw	rt←M[(rs)＋SExt(imm)]	100011	rs	rt	imm
存字sw	M[(rs)＋SExt(imm)]←(rt)	101011	rs	rt	imm
取字节lb	rt←SExt(M[(rs)＋SExt(imm)])	100000	rs	rt	imm
取无符号字节lbu	rt←ZExt(M[(rs)＋SExt(imm)])	100100	rs	rt	imm
存字节sb	M[(rs)＋SExt(imm)]←(rt)$_{7\sim0}$	101000	rs	rt	imm
小于置位slti	rt＝(rs)＜$_{有}$SExt(imm)？1:0	001010	rs	rt	imm
无符号小于置位sltiu	rt＝(rs)＜$_{无}$ZExt(imm)？1:0	000000	rs	rt	imm
相等转移beq	if((rs)＝(rt)) PC←(PC)＋4＋SExt(imm)<<2	000100	rs	rt	imm
不等转移bne	if((rs)≠(rt)) PC←(PC)＋4＋SExt(imm)<<2	000101	rs	rt	imm

思考②：if ($4<$5)、if ($4≥$5)功能如何实现？

```
slt $6, $4, $5
bne $6, $0, LL1
```

跳转j	PC←(PC)$_{高4位}$ ‖ Addr<<2	000010	addr
过程调用jal	$31←(PC)，PC←(PC)$_{高4位}$ ‖ Addr<<2	000011	addr

例1—分别写出实现$1←($2)＋($3)、$1←($2)－2功能的指令字，其中运算均为有符号运算

解—指令字1为：000000 00010 00011 00001 00000 100000

指令字2为：001000 00010 00001 11111 11111 111110

例2—若整型数组A的首址放在$10中，sizeof(int)＝4B，说明下列代码序列的功能：

00000020H：100011 01010 01011 00000 00000 001000

00000024H：001000 01011 01100 00000 00000 001000

00000028H：101011 01010 01100 00000 00000 001000

解—指令功能分别为：

$11←M[($10)＋8]，$12←($11)＋8，M[($10)＋8]←($12)

因($10)＋8＝&A[0＋8/sizeof(int)]＝&A[2]，

故代码功能为A[2]＝A[2]＋8

2、IA32指令系统

***数据表示：**

有整数、浮点数、指针、压缩SIMD数、BCD数、位域
（同MIPS） （32/38位） （64位） （8位） （组合）

***OPD存放：**

存储器—按字节编址、32位地址，小端、对齐方式

IR—可存放8/16/32位立即数，小端、不对齐方式

寄存器—GPRs为8个×32位，可存放8/16/32位OPD

FPRs为8个×32位，双精度OPD占用相邻2个FPR

状态REG有EFLAG(含ZF/CF/SF/OF/AF等标志)

*寻址方式：

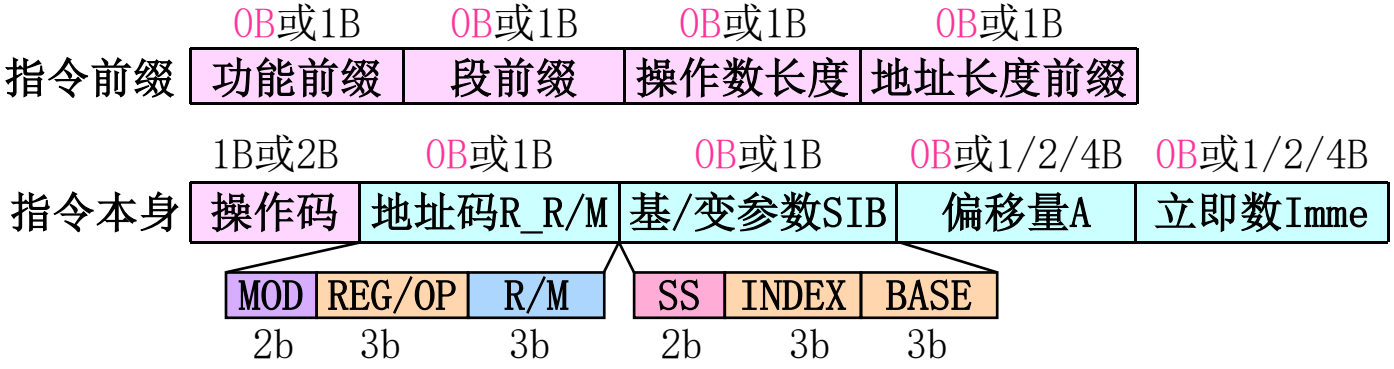
寻址方式	地址形成方法	OPD存放部件
立即寻址	OPD＝指令中的常数	指令寄存器
寄存器寻址	OPD＝(R)	寄存器
直接寻址	EA＝A	存储器
寄存器间接寻址	EA＝(B)或(I)	
寄存器相对寻址(基址寻址)	EA＝(B)或(I)＋A	
基址加变址寻址	EA＝(B)＋(I)	
相对基址加变址寻址	EA＝(B)＋(I)＋A	
比例变址寻址	EA＝(B)＋(I)×S＋A	
相对寻址	EA＝(EIP)＋A	
说明：通用寄存器R—EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI， 基址寄存器B—EBX、EBP，变址寄存器I—ESI、EDI， A—偏移量，EIP—程序计数器，S—比例因子		

数据寻址方式—除相对寻址外的寻址方式，隐含方式

指令寻址方式—直接、寄存器间接、相对等，隐含方式

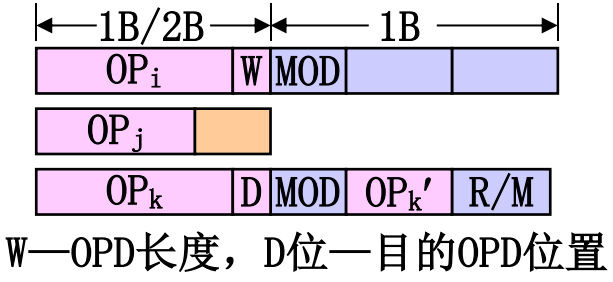
*指令格式：有多种，采用变长指令字结构(1B-16B)

组成—[指令前缀+]指令本身 ←[ ]表示可以缺省

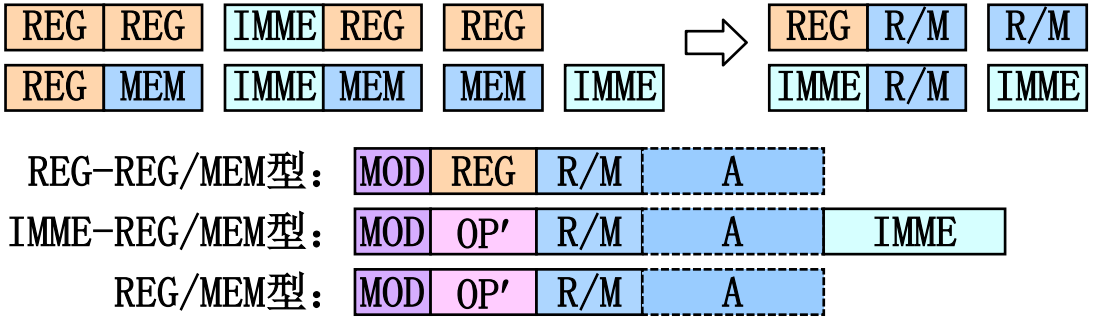


指令前缀—编码与指令本身无二义性，可缺省(当前指令与上条指令相同时) ←保持软件兼容性常见做法 ←前缀保存到系统寄存器中

指令本身—操作码采用扩展编码，地址码(要求≤2个、MEM型≤1个)组合有4种 ←不含寻址方式位



W—OPD长度，D位—目的OPD位置



§4.5 指令系统的发展

※主要内容：CISC、RISC的特征

*系统性能对指令系统的要求：$T_{CPU}=I_N\times CPI\times T_C$

减少 I_N —应增加指令的功能及种类

减少CPI—应减少指令的功能及种类

} 要求矛盾（无法调和）

*指令系统的发展：

面向软件—强化指令的功能及种类 →CISC(Complex Instruction Set Computer)

面向硬件—简化指令的功能及种类 →RISC(Reduced Instruction Set Computer)

1、CISC指令系统

*目标：侧重增强指令功能，以减少程序中的指令数(I_N)

*示例：IA32有200多条标量＋300多SSE指令，8个GPR

*特征：
- 变长指令字结构，指令种类多、格式复杂、寻址方式多
- 大多为REG-MEM型指令，执行时间较长(几个～几十个T_C)
- 指令功能不均衡，编译程序复杂，执行控制复杂，不利于流水等技术

2、RISC指令系统

*目标：侧重简化指令功能，以提高指令执行速度(CPI)

*示例：MIPS32有31条常用指令、32个GPR

*特征：
- 定长指令字结构，指令种类少、格式简单、寻址方式少
- 除Load/Store外，均为REG-REG型指令(执行速度快[=1T_C])
- 使用大量寄存器，便于编译优化、过程调用的快速实现
- 执行控制简单，有利于流水、并行处理技术

3、CISC与RISC的发展

*CISC与RISC的比较：

硬件方面—RISC较好，得益于流水、并行技术

└速度、成本、功耗等方面

软件方面—CISC较好，得益于已有软件、软件兼容性

└开发成本、代码效率、兼容性等方面

性/价方面—相当，CISC成本低、更新快(市场份额大)

*CISC与RISC自身的发展：技术交融、和平共处

RISC—ISA日趋复杂，纯RISC→类RISC；嵌入式方向

└应用领域扩大、为保持性能

CISC—汲取RISC技术，纯CISC→RISC内核；台式机/服务器方向

(如:Intel 80486起)

作业4-2：P166—8、9、11